

Proceduralna generacja krajobrazu - Implementacja

Arseni Karokhin

5 marca 2026

Spis treści

1	Wprowadzenie	2
2	Struktura projektu	2
3	Konfiguracja generacji terenu	2
4	Generacja mapy wysokości	3
5	Normalizacja mapy wysokości	3
6	Budowa geometrii terenu	3
7	Podział terenu na chunki	3
8	System renderowania	4

1 Wprowadzenie

Celem projektu jest implementacja systemu proceduralnej generacji krajobrazu w języku C++ z wizualizacją wyników przy użyciu graficznego API OpenGL. W przeciwieństwie do tradycyjnego podejścia, w którym model krajobrazu przechowywany jest w postaci wcześniej przygotowanego pliku, w niniejszym systemie powierzchnia generowana jest dynamicznie na podstawie algorytmu oraz zestawu parametrów wejściowych. Dzięki temu przy identycznych parametrach wejściowych oraz tej samej wartości seed system zawsze generuje identyczny krajobraz.

Architektura projektu została zaprojektowana w sposób modułowy. Logika generacji terenu została oddzielona od systemu renderowania. Projekt podzielony jest na kilka głównych modułów znajdujących się w katalogu `src`: warstwę aplikacji (`app`), warstwę infrastrukturalną (`core`), moduł generacji terenu (`generator`), moduł renderowania (`render`) oraz dodatkowe moduły (`asssests`, `shaders`, `doc`).

2 Struktura projektu

Główny punkt wejścia programu znajduje się w pliku `main.cpp`. Plik ten inicjalizuje aplikację oraz uruchamia główną pętlę programu.

Logika działania aplikacji znajduje się w klasie `Application` z katalogu `src/app`. Klasa ta odpowiada za inicjalizację systemu, wczytanie parametrów wejściowych oraz zarządzanie główną pętlą programu. W jej obrębie następuje również inicjalizacja systemu generacji terenu z katalogu `src/generator` oraz systemu renderowania `render`.

W katalogu `src/core` znajdują się elementy infrastrukturalne wykorzystywane w różnych częściach projektu. I będą zdefiniowane już w trakcie realizacji projektu.

3 Konfiguracja generacji terenu

Parametry generacji terenu przechowywane są w strukturze `TerrainConfig`, znajdującej się w pliku `TerrainConfig.hpp`. Struktura ta zawiera wszystkie wartości potrzebne do wygenerowania krajobrazu, takie jak:

1. rozmiar generowanego obszaru, parametr `N` (`size`),
2. odległość pomiędzy punktami siatki (`cellSize`),
3. parametry funkcji szumu (`noiseScale`, liczba oktaf, `persistence`, `lacunarity`),
4. zakres wysokości (`minHeight`, `maxHeight`),
5. parametr poziomu morza (`seaLevel`)
6. parametr `seed`.

Parametr `seed` może być wprowadzony przez użytkownika w dowolnej formie. W systemie jest on przekształcany do wartości liczbowej przy użyciu deterministycznej funkcji haszującej. Dzięki temu możliwe jest używanie zarówno identyfikatorów tekstowych, jak i liczbowych, przy zachowaniu deterministyczności generacji.

Po przetworzeniu parametrów tworzony jest obiekt konfiguracji, który przekazywany jest do systemu generacji terenu.

4 Generacja mapy wysokości

Proces generacji krajobrazu rozpoczyna się od utworzenia mapy wysokości (heightmap). Za generację danych odpowiada klasa `HeightMapGenerator`, której implementacja znajduje się w pliku `HeightMapGenerator.cpp`. W systemie wykorzystywane są dwie struktury danych reprezentujące mapę wysokości:

1. `HeightMapRaw` — przechowuje surowe wartości wygenerowane przez funkcję szumu,
2. `HeightMapFin` — przechowuje wartości po procesie normalizacji i skalowania.

Każdy element mapy wysokości reprezentuje wysokość w danym punkcie siatki.

Dla każdej komórki siatki obliczane są współrzędne przestrzenne punktu. Współrzędne te przekazywane są do funkcji szumu, zaimplementowanej w module `Noise.cpp`. Funkcja ta generuje wartość szumu dla zadanych współrzędnych.

W projekcie wykorzystywana jest metoda fraktalnego szumu znana jako fBm (fractal Brownian motion). Metoda ta polega na sumowaniu kilku warstw szumu o różnych częstotliwościach i amplitudach. Parametry takie jak liczba oktafów, persistence oraz lacunarity kontrolują charakter generowanego krajobrazu.

Wyniki generacji zapisywane są w strukturze `HeightMapRaw`.

5 Normalizacja mapy wysokości

Po wygenerowaniu surowych wartości wysokości przeprowadzany jest proces normalizacji. W tym celu analizowany jest cały zbiór wartości mapy wysokości w celu znalezienia wartości minimalnej oraz maksymalnej.

Następnie każda wartość wysokości przekształcana jest do zakresu od 0 do 1 według wzoru normalizacji. Wynik zapisywany jest w strukturze `HeightMapFin`.

6 Budowa geometrii terenu

Po przygotowaniu mapy wysokości następuje budowa geometrii powierzchni. Proces ten realizowany jest przez klasę `MeshBuilder`, której implementacja znajduje się w pliku `MeshBuilder.cpp`.

Klasa `MeshBuilder` przekształca dane z mapy wysokości w siatkę trójkątów. Dla każdego punktu siatki tworzony jest wierzchołek zawierający współrzędne przestrzenne. Współrzędne poziome obliczane są na podstawie indeksów punktu oraz wartości `cellSize`, natomiast współrzędna pionowa wyznaczana jest na podstawie wartości mapy wysokości.

Powstałe wierzchołki łączone są w trójkąty poprzez utworzenie odpowiednich indeksów. Każda komórka siatki dzielona jest na dwa trójkąty, co pozwala utworzyć ciągłą powierzchnię terenu.

7 Podział terenu na chunki

W celu poprawy wydajności systemu teren dzielony jest na mniejsze fragmenty, zwane chunkami. Za zarządzanie chunkami odpowiadają klasy `ChunkCreator` oraz `ChunkStorage`.

Klasa **ChunkCreator** odpowiada za generowanie pojedynczych fragmentów terenu, natomiast **ChunkStorage** zarządza kolekcją wygenerowanych chunków oraz umożliwia ich efektywne przechowywanie i dostęp.

Takie podejście pozwala na generowanie i renderowanie tylko tych fragmentów krajobrazu, które znajdują się w pobliżu kamery.

8 System renderowania

Wizualizacja terenu realizowana jest przez moduł render. Główną klasą odpowiedzialną za wyświetlanie sceny jest **Renderer**, zaimplementowana w pliku **Renderer.cpp**.

System renderowania wykorzystuje również następujące komponenty:

1. **MeshGPU** — odpowiada za przesyłanie danych geometrii do pamięci karty graficznej,
2. **Shader** — obsługuje programy shaderowe wykorzystywane podczas renderowania,
3. **Texture** — odpowiada za obsługę tekstur powierzchni,
4. **Camera** — odpowiada za pozycję oraz orientację kamery w scenie.

Dane geometrii generowane przez moduł generator są przekazywane do **MeshGPU**, gdzie zapisywane są w buforach GPU. Następnie klasa **Renderer** wykorzystuje te dane do wyświetlenia sceny przy użyciu odpowiednich shaderów.